22 DE OCTUBRE DE 2024

PROYECTO MEDHEART CAPSTONE 2024

EVIDENCIAS DEL PROYECTO

JAVIERA LIZANA SALAZAR – MAXIMILIANO LASMAR ROSSIER
INSTITUTO PROFESIONAL DUOC UC
Sede Viña del Mar

Contenido

Base de datos - Google Cloud Firebase Firestore 2

Evidencia base de datos Firebase 3

Backend – Vertex AI Workbench GCP 4

Evidencia Vertex AI Workbench GCP 5

Dataset Diagnóstico 6

Evidencia Dataset Diagnóstico 8

Algoritmo Machine Learning Random Forest 9

Evidencia Algoritmo Random Forest 10

Backend – Flask / Python / Pickle 11

Evidencia Flask / Python 12

Seguridad – Firewall GCP 13

Evidencia de seguridad Firewall GCP 14

Netlify 15

Evidencia Netlify 16

Gestión de Proyecto Trello 17

Evidencia Tello 18

Postman 19

Evidencia Postman 20

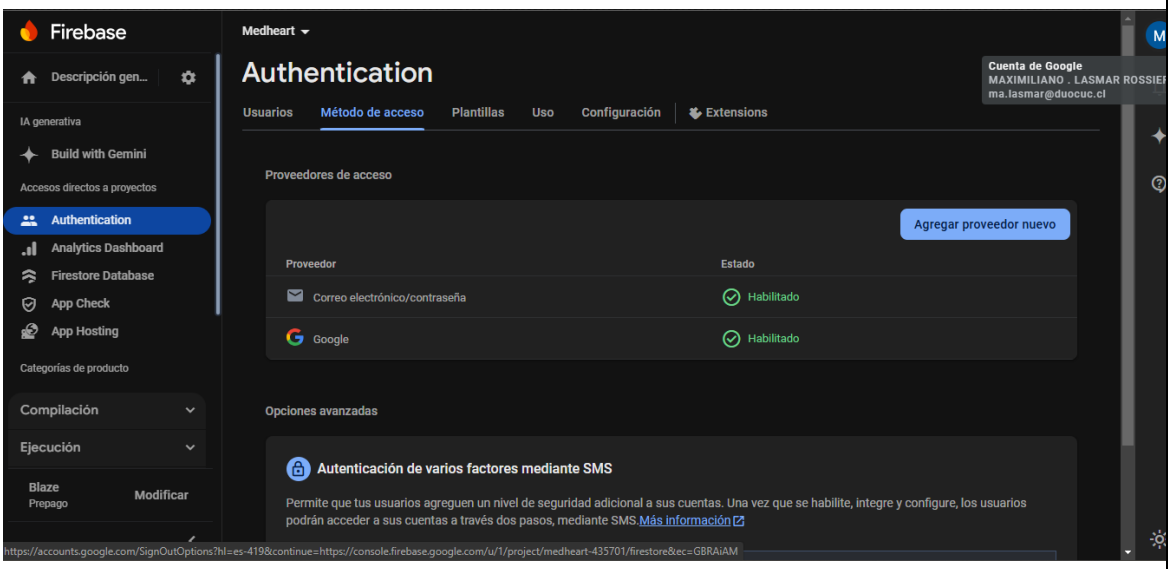
Base de datos - Google Cloud Firebase Firestore

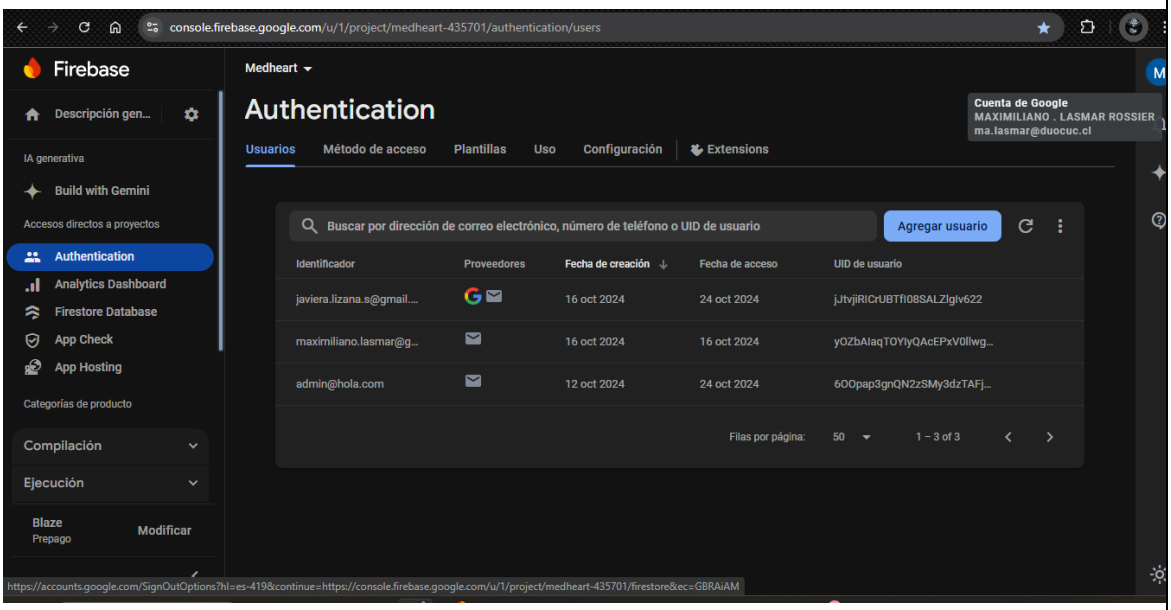
En el proyecto MedHeart, utilizamos Google Cloud Firebase Firestore como nuestra base de datos principal para manejar la autenticación de usuarios en el sistema de login. Firestore, al ser una base de datos no relacional de GCP, nos proporciona flexibilidad en el manejo de datos dinámicos, como los perfiles de usuarios y sus credenciales. Esto nos permite escalar de manera eficiente sin la rigidez de las estructuras relacionales, facilitando el almacenamiento de documentos y colecciones con diferentes tipos de información, lo que se alinea perfectamente con la naturaleza cambiante de nuestro proyecto.

Además, una de las principales razones por las que seleccionamos Firestore es por los créditos otorgados por Duoc UC, que nos permiten aprovechar los servicios de GCP sin costos adicionales durante el desarrollo. Esto nos brinda acceso a un sistema robusto y altamente disponible, ideal para un entorno de producción.

En comparación con una base de datos relacional, Firestore ofrece ventajas en términos de rendimiento y escalabilidad para aplicaciones que requieren alta disponibilidad y manejan datos heterogéneos, como es el caso de MedHeart. La naturaleza no relacional de Firestore simplifica el manejo de datos distribuidos, lo que es crítico en nuestra aplicación web y móvil.

Evidencia base de datos Firebase





Backend – Vertex AI Workbench GCP

En el proyecto MedHeart, hemos decidido utilizar Vertex AI Workbench de Google Cloud para desarrollar y entrenar el algoritmo de Random Forest destinado al diagnóstico de enfermedades cardíacas. Vertex AI Workbench nos proporciona un entorno altamente flexible y escalable para trabajar con grandes volúmenes de datos y realizar tareas intensivas en cómputo, como el entrenamiento de modelos de machine learning.

Dentro de este entorno, hemos creado una instancia llamada medheartdiagnostico, en la cual utilizamos Jupyter Lab para escribir, ejecutar y depurar el código del algoritmo Random Forest. Este entorno nos permite acceder fácilmente a los recursos de Google Cloud, como almacenamiento y potencia de procesamiento, optimizando la carga y ejecución de nuestro dataset de diagnóstico, el cual contiene datos clínicos utilizados para predecir enfermedades cardíacas.

La elección de Vertex AI Workbench nos ofrece una infraestructura gestionada y adaptable, crucial para el manejo de datasets grandes y complejos. Además, al integrarse sin problemas con otras herramientas de GCP que ya estamos utilizando en el proyecto, nos permite centralizar todo el flujo de trabajo de machine learning, desde la experimentación hasta la implementación final.

Evidencia Vertex AI Workbench GCP

console.cloud.google.com/iam-admin/iam?authuser=1&organizationId=527750334036&project=medheart-435701

Google Cloud | Medheart | Buscar (/) recursos, documentos, productos y más | Buscar

IAM y administración

- IAM
- PAM
- Límite de acceso principal
- Organizaciones ...
- Identidad y organización
- Solucionador de problem...
- Analizador de políticas
- Políticas de la organización
- Cuentas de servicio
- Federación de identidade
- Administrar recursos
- Notas de versión

IAM | APRENDIZAJE

PERMITIDA | RECHAZAR | HISTORIAL DE RECOMENDACIONES

Permisos para el proyecto "Medheart"

Estos permisos afectan a este proyecto y todos sus recursos. Más información

Include Google-provided role grants

VER POR PRINCIPALES | VER POR ROLES

OTORGAR ACCESO | QUITAR ACCESO

Filtro | Ingresar el nombre o el valor de la propiedad

Tipo	Principal	Nombre	Rol	
	743108577727-compute@developer.gserviceaccount.com	Compute Engine default service account	Administrador de Compute	Estadi
			Administrador de instancias de Compute (v1)	Estadi
			Administrador de notebooks	Estadi
			Administrador de políticas de firewall de Compute en la organización	Estadi
			Administrador de Service Networking	Estadi
			Agente de servicios de Compute Engine	Estadi
			Editor	8955/8

console.cloud.google.com/vertex-ai/workbench/locations/us-central1-a/instances/medheartdiagnostico?authuser=1&organizationId=527750334036&project...

Google Cloud | Medheart | Buscar (/) recursos, documentos, productos y más | Buscar

Vertex AI

TOOLS

- Panel
- Model Garden
- Canalizaciones

NOTEBOOKS

- Colab Enterprise
- Workbench

VERTEX AI STUDIO

- Descripción general
- Formato libre
- Chat
- Marketplace

Detalles de la ... | ABRIR JUPYTERLAB | INICIAR | DETENER | RESTABLECER

Cuenta: MAXIMILIANO . LASMAR ROSSIER (ma.lasmar@duocuc.cl)

Hay una actualización disponible para esta instancia. Más información

medheartdiagnostico

Estado	Zona	Tipo de máquina	GPU	Propietario
Activo	us-central1-a	e2-standard-2 (Efficient Instance: 2 CPU virtuales, 8 GB de RAM)	Ninguna	743108577727-compute@developer.gserviceaccount.com

SISTEMA | HARDWARE | SOFTWARE Y SEGURIDAD | ESTADO | SUPERVISIÓN | REGISTROS

Versión del entorno	M125
Fecha de creación	12 oct 2024, 3:21:56 p.m.
Última modificación	26 oct 2024, 7:15:33 p.m.
Copia de seguridad	Sin especificar
Subred	default
Detalles de la VM	Ver en Compute Engine

Historial de actualizaciones

Incluye todas las actualizaciones de entorno realizadas para esta instancia. Si es

Dataset Diagnóstico

El dataset de diagnóstico utilizado en el proyecto MedHeart contiene una amplia gama de datos clínicos esenciales para la predicción de enfermedades cardíacas. Este conjunto de datos incluye variables clave como la edad, el género, los niveles de colesterol, la presión arterial, los resultados de electrocardiogramas, entre otros. Estas características permiten capturar una representación detallada del estado de salud cardiovascular de los pacientes, lo que es fundamental para identificar patrones y correlaciones que puedan llevar a un diagnóstico preciso.

La elección de este dataset está justificada por su capacidad para representar diversos factores de riesgo asociados con enfermedades cardíacas, lo que lo convierte en una fuente confiable y robusta para el entrenamiento de nuestro modelo de Random Forest. Al incluir tanto características continuas como categóricas, el dataset facilita que el algoritmo maneje la complejidad inherente de los datos médicos, lo que mejora la capacidad de predicción y precisión del modelo.

Además, este dataset ha sido diseñado para abordar tanto la presencia como la gravedad de las enfermedades cardíacas, lo que es fundamental para el objetivo del proyecto MedHeart de proporcionar diagnósticos detallados y confiables. Esto nos permite desarrollar una herramienta de predicción que no solo identifica la posible presencia de una enfermedad cardíaca, sino que también ayuda a los médicos a entender qué factores contribuyen más al diagnóstico, mejorando la toma de decisiones clínicas.

Estructura del dataset:

- age: Edad del paciente

- sex: Género (1 para masculino, 0 para femenino)

- cp: Tipo de dolor de pecho

- trestbps: Presión arterial en reposo

- chol: Colesterol sérico

- fbs: Nivel de azúcar en sangre en ayunas (> 120 mg/dl)

- restecg: Resultados del electrocardiograma en reposo

- thalach: Frecuencia cardíaca máxima alcanzada

- exang: Angina inducida por ejercicio

- oldpeak: Depresión del ST inducida por el ejercicio

- slope: Pendiente del segmento ST

- ca: Número de vasos principales coloreados por fluoroscopia

- thal: Tasa de talasemia

- target: Resultado del diagnóstico de enfermedad cardíaca (0 o 1)

Este dataset será utilizado para entrenar el algoritmo Random Forest en el proyecto MedHeart, ayudando a predecir enfermedades cardíacas basadas en estos factores clínicos. La variable objetivo "target" indica la presencia o ausencia de enfermedad cardíaca, y el resto de las columnas son características clínicas que el algoritmo Random Forest utilizará para hacer predicciones.

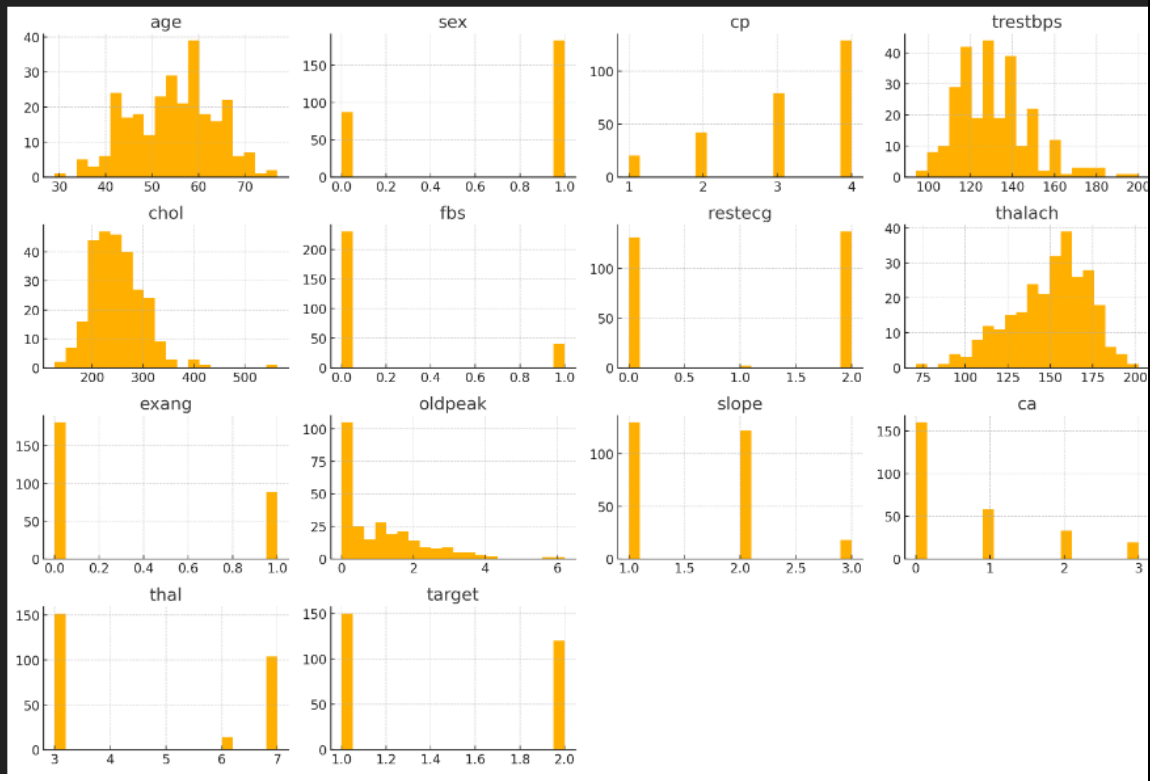
Evidencia Dataset Diagnóstico

Resultado

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak
0	70.0	1.0	4.0	130.0	322.0	0.0	2.0	109.0	0.0	2.4
1	67.0	0.0	3.0	115.0	564.0	0.0	2.0	160.0	0.0	1.6
2	57.0	1.0	2.0	124.0	261.0	0.0	0.0	141.0	0.0	0.3
3	64.0	1.0	4.0	128.0	263.0	0.0	0.0	105.0	1.0	0.2
4	74.0	0.0	2.0	120.0	269.0	0.0	2.0	121.0	1.0	0.2

	slope	ca	thal	target
0	2.0	3.0	3.0	2
1	2.0	0.0	7.0	1
2	1.0	0.0	7.0	2
3	2.0	1.0	7.0	1
4	1.0	1.0	3.0	1

Matplotlib Chart



Algoritmo Machine Learning Random Forest

En el proyecto MedHeart, hemos optado por utilizar el algoritmo Random Forest para analizar y clasificar los datos del diagnóstico de enfermedades cardíacas debido a su capacidad de manejar datos complejos y multivariables de manera eficaz. Este dataset incluye características clave del estado de salud de los pacientes, como la edad, el colesterol, la presión arterial y los resultados de electrocardiogramas, lo que requiere un modelo capaz de manejar tanto datos continuos como categóricos.

El algoritmo Random Forest es especialmente adecuado para este tipo de tarea porque genera múltiples árboles de decisión, cada uno entrenado con diferentes subconjuntos del dataset. Al promediar los resultados de estos árboles, Random Forest minimiza el riesgo de sobreajuste, lo que mejora la generalización del modelo para nuevos datos clínicos. Esto es crucial en el contexto médico, donde es fundamental que el modelo no solo funcione bien en los datos de entrenamiento, sino que también sea capaz de predecir con precisión en casos reales.

Otra ventaja del uso de Random Forest es su capacidad para manejar la importancia de cada característica en el diagnóstico, lo que permite a los médicos entender qué factores son más relevantes en la predicción de una enfermedad cardíaca. Esto mejora la interpretabilidad del modelo, un aspecto fundamental para ganar confianza en las herramientas de diagnóstico asistido por inteligencia artificial. Por estas razones, hemos seleccionado Random Forest como la mejor opción para construir un modelo predictivo que mejore la precisión del diagnóstico en el proyecto MedHeart.

Evidencia Algoritmo Random Forest

6b25f90419174caa-dot-us-central1.notebooks.googleusercontent.com/lab/tree/MedheatDiagnostico.ipynb

medheartdiagnostico

File Edit View Run Kernel Git Tabs Settings Help

e2-standard-2

app.py | MedheatDiagnostico.ipynb | Terminal 2

Code | Execute | Python 3 (ipykernel) (Local)

Filter files by name

/

Name

- app.py
- Diagnostico.ipynb
- Diagnostico2.ipynb
- heart_disease_rando...
- heart.dat
- heart.doc
- Index
- MedheatDiagnostico....
- modelo_random_for...
- notebook_template.i...
- scaler.pkl
- Untitled1.ipynb

[2]:
```python
# Importar las bibliotecas necesarias
import pandas as pd
import numpy as np
import pickle
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix, roc_auc_score
from imblearn.over_sampling import SMOTE
from sklearn.calibration import CalibratedClassifierCV

# 1. Cargar el dataset
columns = ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
           'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target']

# Leer el archivo .dat y convertir todo a numérico
data = pd.read_csv('heart.dat', sep='\s+', names=columns)
data = data.apply(pd.to_numeric, errors='coerce')

# Eliminar filas con valores nulos
data = data.dropna()

# Separar las características (X) y la variable objetivo (y)
X = data.drop('target', axis=1)
y = data['target']
```

Simple | Python 3 (ipykernel) (Local) | Idle | Mode: Command | Ln 1, Col 1 | MedheatDiagnostico.ipynb

6b25f90419174caa-dot-us-central1.notebooks.googleusercontent.com/lab/tree/MedheatDiagnostico.ipynb

medheartdiagnostico

File Edit View Run Kernel Git Tabs Settings Help

e2-standard-2

app.py | MedheatDiagnostico.ipynb | Terminal 2

Code | Execute | Python 3 (ipykernel) (Local)

Filter files by name

/

Name

- app.py
- Diagnostico.ipynb
- Diagnostico2.ipynb
- heart_disease_rando...
- heart.dat
- heart.doc
- Index
- MedheatDiagnostico....
- modelo_random_for...
- notebook_template.i...
- scaler.pkl
- Untitled1.ipynb

```
Mejores hiperparámetros: {'bootstrap': True, 'max_depth': 5, 'max_features': 'sqrt', 'min_samples_leaf': 4, 'min_samples_split':
5, 'n_estimators': 200}
Accuracy con umbral ajustado: 0.8703703703703703
              precision    recall  f1-score   support

           0       0.88      0.91      0.90        33
           1       0.85      0.81      0.83        21

    accuracy                           0.87        54
   macro avg       0.87      0.86      0.86        54
weighted avg       0.87      0.87      0.87        54

Matriz de confusión:
[[30  3]
 [ 4 17]]
Accuracy promedio en validación cruzada: 0.8465309898242367
ROC-AUC Score: 0.9148629148629149
Modelo guardado exitosamente en 'modelo_random_forest_calibrado.pkl'
[CV] END bootstrap=False, max_depth=15, max_features=log2, min_samples_leaf=4, min_samples_split=5, n_estimators=300; total time=
0.6s
[CV] END bootstrap=False, max_depth=15, max_features=log2, min_samples_leaf=4, min_samples_split=5, n_estimators=500; total time=
1.0s
[CV] END bootstrap=False, max_depth=15, max_features=log2, min_samples_leaf=4, min_samples_split=5, n_estimators=500; total time=
0.9s
[CV] END bootstrap=False, max_depth=15, max_features=log2, min_samples_leaf=4, min_samples_split=5, n_estimators=500; total time=
0.9s
[CV] END bootstrap=False, max_depth=15, max_features=log2, min_samples_leaf=4, min_samples_split=10, n_estimators=200; total time
```

Simple | Python 3 (ipykernel) (Local) | Idle | Mode: Command | Ln 1, Col 1 | MedheatDiagnostico.ipynb

Backend – Flask / Python / Pickle

En el proyecto MedHeart, hemos decidido utilizar Flask junto con Python y la librería Pickle para manejar el despliegue del modelo de diagnóstico basado en el algoritmo Random Forest. Esta combinación de herramientas nos permite desarrollar una arquitectura eficiente y flexible para la aplicación, facilitando tanto la integración del Backend como la interoperabilidad con el frontend.

Flask es un microframework ligero y escalable que nos permite construir aplicaciones web de manera rápida y sencilla. Lo hemos elegido por su capacidad para manejar solicitudes HTTP, lo que nos permite integrar de manera efectiva el modelo de diagnóstico con la interfaz de usuario, desarrollada en React. Flask es ideal para nuestro proyecto porque nos permite crear una API RESTful que conecta el modelo de machine learning con la interfaz, donde los usuarios pueden enviar datos clínicos y recibir los resultados del diagnóstico de enfermedades cardíacas.

Python ha sido nuestra elección principal para el desarrollo del modelo de diagnóstico, ya que ofrece una amplia variedad de bibliotecas para el procesamiento de datos, machine learning y análisis. Además, Python es una herramienta ampliamente utilizada en la comunidad de desarrollo de IA y tiene un soporte robusto para algoritmos como Random Forest, que hemos utilizado para nuestro modelo predictivo. Su versatilidad y simplicidad nos permiten integrar el código del modelo fácilmente dentro de Flask.

Pickle es una herramienta crucial para nuestro flujo de trabajo, ya que nos permite serializar (guardar) el modelo de Random Forest entrenado en un archivo. Esto es fundamental porque, una vez que el modelo ha sido entrenado con el dataset de diagnóstico, necesitamos guardarlo para su uso posterior sin tener que volver a entrenarlo cada vez que se inicie el servidor. Con Pickle, podemos cargar el modelo pre entrenado rápidamente en Flask y hacer predicciones en tiempo real a partir de los datos clínicos ingresados por los usuarios.

En conjunto, Flask, Python y Pickle nos proporcionan una solución robusta, escalable y eficiente para desplegar el modelo de diagnóstico de MedHeart, permitiendo una interacción fluida entre los médicos, el sistema y el modelo de inteligencia artificial, garantizando una experiencia de usuario óptima.

Evidencia Flask / Python

6b25f90419174caa-dot-us-central1.notebooks.googleusercontent.com/lab/tree/app.py

medheartdiagnostico

File Edit View Run Kernel Git Tabs Settings Help

e2-standard-2

app.py | MedheatDiagnostico.ipynb | Terminal 2

Filter files by name

/

Name

- app.py
- Diagnostico.ipynb
- Diagnostico2.ipynb
- heart_disease_rando...
- heart.dat
- heart.doc
- Index
- MedheatDiagnostico....
- modelo_random_for...
- notebook_template.i...
- scaler.pkl
- Untitled1.ipynb

```python
from flask import Flask, request, jsonify
import pickle
import numpy as np
import pandas as pd
from flask_cors import CORS

# Cargar el modelo entrenado y el escalador
with open('modelo_random_forest_calibrado.pkl', 'rb') as model_file:
    modelo = pickle.load(model_file)

# Cargar el escalador guardado
with open('scaler.pkl', 'rb') as scaler_file:
    scaler = pickle.load(scaler_file)

# Inicializar la aplicación Flask
app = Flask(__name__)
CORS(app, resources={r"/api/*": {"origins": "*"}})  # Permitir todas las solicitudes

@app.route('/api/predict', methods=['POST'])
def predict():
    # Recibir los datos del formulario como JSON
    data = request.json

    # Convertir los datos a formato numpy array
    input_data = np.array([[data['age'], data['sex'], data['cp'], data['trestbps'],
                            data['chol'], data['fbs'], data['restecg'], data['thalach'],
                            data['exang'], data['oldpeak'], data['slope'], data['ca'],
                            data['thal']]])
```

Simple 1 1 Python Ln 1, Col 1 Spaces: 4 app.py 2

6b25f90419174caa-dot-us-central1.notebooks.googleusercontent.com/lab

medheartdiagnostico

File Edit View Run Kernel Git Tabs Settings Help

e2-standard-2

app.py | MedheatDiagnostico.ipynb | Terminal 2

Filter files by name

/

Name

- app.py
- Diagnostico.ipynb
- Diagnostico2.ipynb
- heart_disease_rando...
- heart.dat
- heart.doc
- Index
- MedheatDiagnostico....
- modelo_random_for...
- notebook_template.i...
- scaler.pkl
- Untitled1.ipynb

```
(base) jupyter@medheartdiagnostico:~$ python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://10.128.0.5:5001
Press CTRL+C to quit
```

Simple 1 1 Terminal 2 2

Seguridad – Firewall GCP

En el proyecto MedHeart, hemos configurado una regla de firewall específica para permitir el acceso seguro a nuestra instancia de diagnóstico, alojada en Google Cloud Platform. Esta regla, denominada “permitir-puerto-5001”, está configurada para permitir el tráfico entrante en el puerto TCP 5001, el cual es utilizado por nuestra API Flask para manejar las solicitudes relacionadas con el diagnóstico de enfermedades cardíacas.

La justificación de esta regla de firewall es fundamental para garantizar que la instancia de la aplicación pueda recibir solicitudes HTTP desde el frontend y procesar los datos clínicos de los usuarios en tiempo real. Dado que el puerto 5001 está asociado a nuestra API, habilitarlo es esencial para asegurar que el flujo de datos entre los usuarios y el modelo de diagnóstico se realice de manera fluida y sin interrupciones.

Además, la implementación de esta regla está configurada para asegurar que solo se permita el tráfico necesario, lo que mejora la seguridad de la instancia al bloquear accesos no deseados. Esto es crucial en un proyecto como MedHeart, donde la privacidad y seguridad de los datos médicos son prioridades. Al restringir el acceso al puerto específico y asegurar que solo el tráfico legítimo pueda llegar a la instancia, mitigamos el riesgo de ataques y accesos no autorizados a los sistemas críticos.

Evidencia de seguridad Firewall GCP

console.cloud.google.com/net-security/firewall-manager/firewall-policies/list?authuser=1&organizationId=527750334036&project=medheart-435701

Google Cloud | Medheart | Buscar (/) recursos, documentos, productos y más | Buscar

Seguridad de red

- Proxy web seguro
- Cloud Armor
  - Panel de DDoS
  - Políticas de Cloud Armor
  - Protección adaptable
  - Nivel de servicio de Cloud...
- IDS de Cloud
  - Panel del IDS
  - Extremos de IDS
  - Amenazas en IDS
- Cloud NGFW
  - Panel

Políticas de firewall | CREAR POLÍTICA DE FIREWALL | CREAR REGLA DE FIREWALL

Cuenta: MAXIMILIANO . LASMAR ROSSIER (ma.lasmar@duocuc.cl)

Reglas de firewall de VPC

Las reglas de firewall controlan el tráfico saliente o entrante de una instancia. De forma predeterminada, se bloquea el tráfico que proviene del exterior de tu red. Más información

Nota: Los firewalls de App Engine se administran en Sección de reglas de firewall de App Engine.

El puerto SMTP 25 no está autorizado en este proyecto. Más información

ACTUALIZAR | CONFIGURAR REGISTROS | BORRAR

Filtro: Ingresar el nombre o el valor de la propiedad

Nombre	Tipo	Destinos	Filtros	Protocolos/puertos	Acción	Prioridad	Red	Registros
permitir-puerto-5001	Entrada	Aplicar a	Intervalos	tcp:5001	Permitir	1000	default	Desactiv
default-allow-icmp	Entrada	Aplicar a	Intervalos	icmp	Permitir	65534	default	Desactiv
default-allow-internal	Entrada	Aplicar a	Intervalos	tcp:0-65535 udp:0-65535 icmp	Permitir	65534	default	Desactiv

console.cloud.google.com/net-security/firewall-manager/firewall-policies/details/permitir-puerto-5001?project=medheart-435701&authuser=1&organiz...

Google Cloud | Medheart | Buscar (/) recursos, documentos, productos y más | Buscar

Seguridad de red

- Proxy web seguro
- Cloud Armor
  - Panel de DDoS
  - Políticas de Cloud Armor
  - Protección adaptable
  - Nivel de servicio de Cloud...
- IDS de Cloud
  - Panel del IDS
  - Extremos de IDS
  - Amenazas en IDS
- Cloud NGFW
  - Panel

Protocolos y puertos
tcp:5001

Aplicación
Habilitada

Estadísticas
Ninguno

Supervisión de aciertos
–

Aplicable a las instancias

En la siguiente tabla, no se muestran instancias de entorno flexible de App Engine

Filtro: Filtrar según nombre de instancia, proyecto o subred

Nombre	Subred	Rangos de IP internas	Rangos de IP externas	Etiquetas	Cuentas de servicio
medheartdiagnostico	default	10.128.0.5	Ninguno	deeplearning-	743108577727-

REST EQUIVALENTE

Netlify

En el proyecto MedHeart, hemos decidido utilizar Netlify como plataforma de despliegue para el frontend de la aplicación, debido a su simplicidad, eficiencia y características avanzadas que se ajustan a las necesidades de nuestro proyecto. Netlify nos permite implementar fácilmente la interfaz web desarrollada en React, lo que garantiza una experiencia de usuario fluida y accesible desde cualquier dispositivo.

Una de las principales razones para elegir Netlify es su capacidad de integrar de manera sencilla con nuestro repositorio de código en GitHub, lo que facilita el flujo de trabajo continuo y automatizado. Cada vez que realizamos cambios en el código, Netlify se encarga de generar y desplegar una nueva versión de la aplicación de manera automática, lo que ahorra tiempo y reduce errores en el proceso de despliegue.

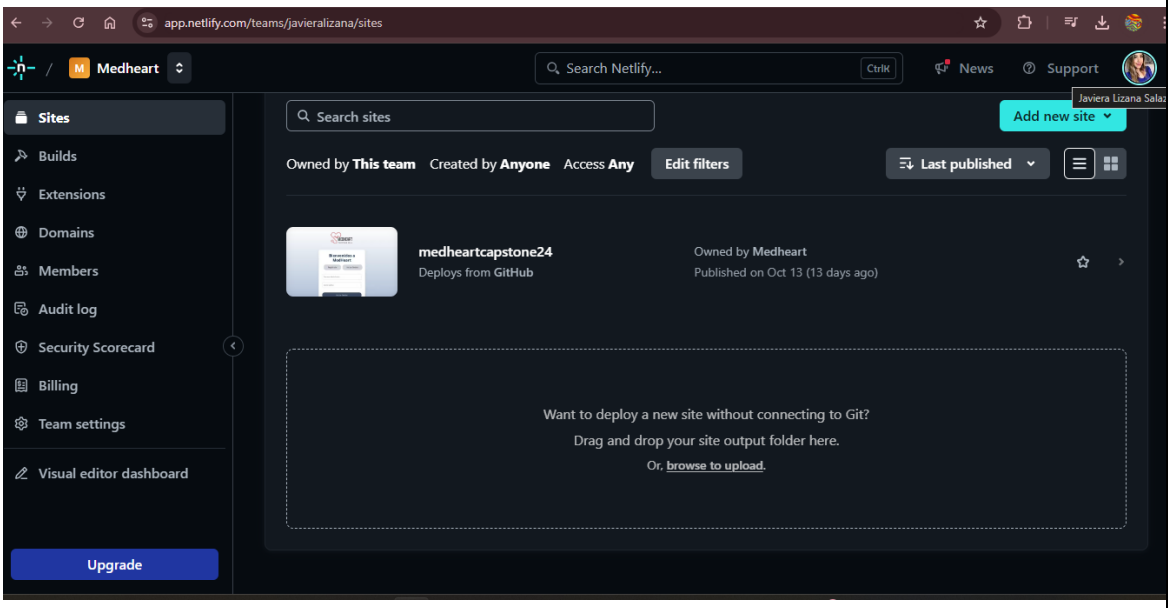
Además, Netlify proporciona HTTPS automáticamente, lo que mejora la seguridad de la aplicación al encriptar las comunicaciones entre el navegador del usuario y el servidor. Esto es especialmente relevante en el proyecto MedHeart, donde se manejan datos médicos sensibles, y es crucial asegurar la confidencialidad y protección de la información.

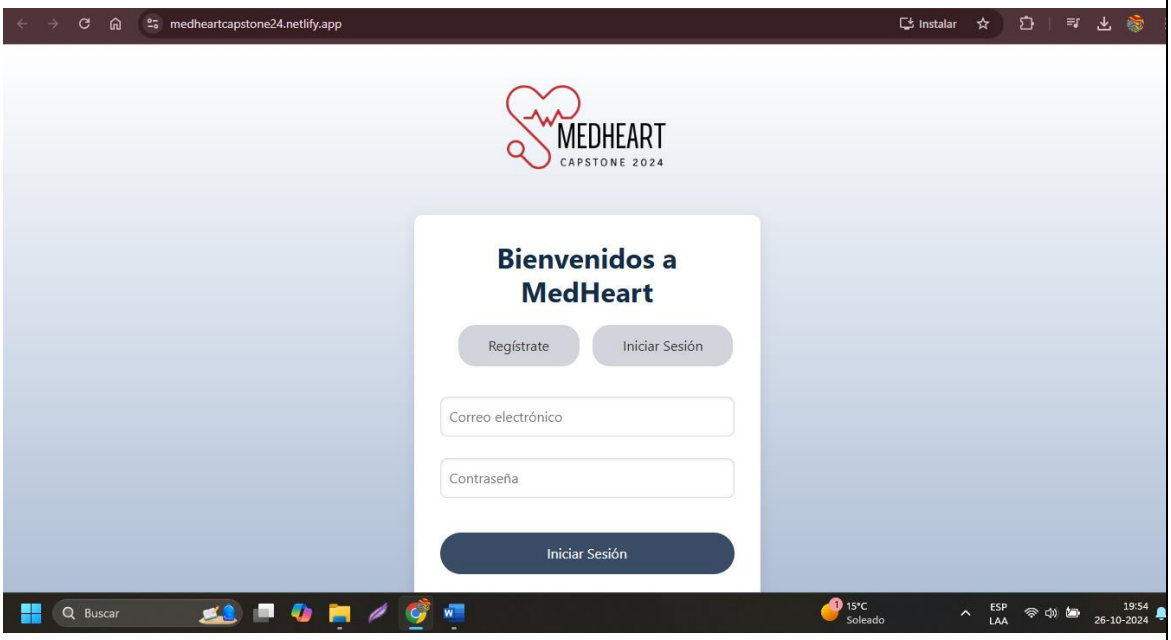
Otro aspecto importante es la velocidad y el rendimiento que ofrece Netlify, ya que utiliza una red de distribución de contenido (CDN) que permite que la aplicación se cargue rápidamente en cualquier parte del mundo, brindando una mejor experiencia a los usuarios. También hemos aprovechado su funcionalidad de serverless functions para manejar algunas lógicas del backend de manera eficiente.

En resumen, Netlify es una solución completa para el despliegue del frontend del proyecto MedHeart, ofreciendo integración continua, seguridad, rendimiento y escalabilidad, aspectos fundamentales para el éxito de nuestra plataforma de diagnóstico de enfermedades cardíacas.

Evidencia Netlify

https://app.netlify.com/teams/javieralizana/sites





Gestión de Proyecto Trello

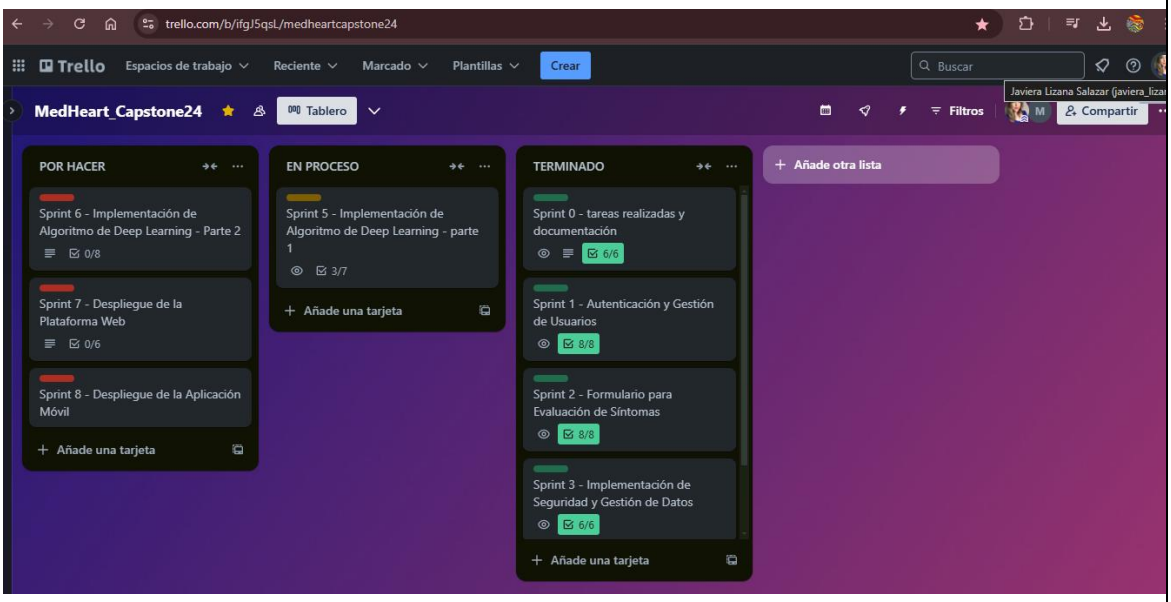
En el proyecto MedHeart, hemos elegido utilizar Trello como nuestra herramienta principal de gestión de tareas y seguimiento de progreso debido a su facilidad de uso, flexibilidad y capacidad para adaptarse a la metodología ágil SCRUM que estamos empleando. Trello nos permite organizar de manera visual y clara las tareas de cada sprint, asegurando que todo el equipo esté alineado con los objetivos y los tiempos de entrega.

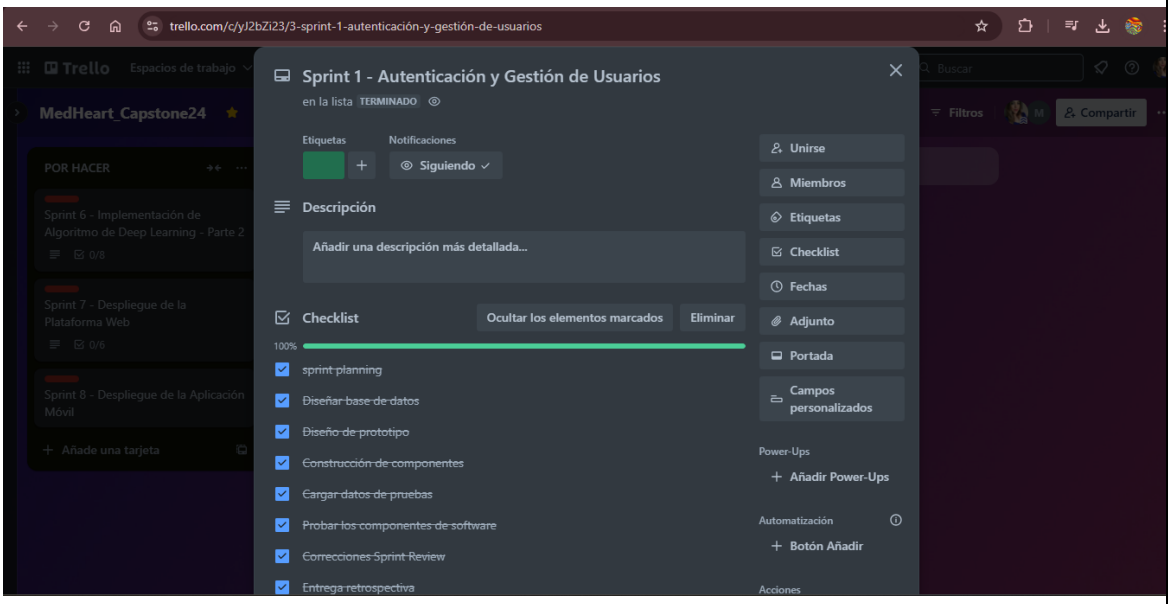
La interfaz intuitiva de Trello, basada en tableros y tarjetas, nos permite desglosar las tareas en épicas, historias de usuario y subtareas, proporcionando una visión clara de las prioridades y el estado de cada actividad. Cada tarjeta puede asignarse a miembros del equipo, lo que facilita la colaboración y asegura que cada integrante sepa qué debe hacer y cuándo debe hacerlo.

En el contexto de SCRUM, Trello es esencial para visualizar y ajustar el backlog del producto, gestionar el progreso de cada sprint mediante columnas como “Por Hacer”, “En Progreso”, “Terminado”, y hacer un seguimiento efectivo de los burndown charts. Esto nos permite identificar cuellos de botella o tareas retrasadas y actuar rápidamente para mantener el ritmo del proyecto.

Trello ha sido una herramienta crucial para gestionar de manera eficiente el flujo de trabajo y la colaboración en el proyecto MedHeart, proporcionando una plataforma ágil y accesible para coordinar a todo el equipo y asegurar el cumplimiento de los plazos establecidos.

Evidencia Tello





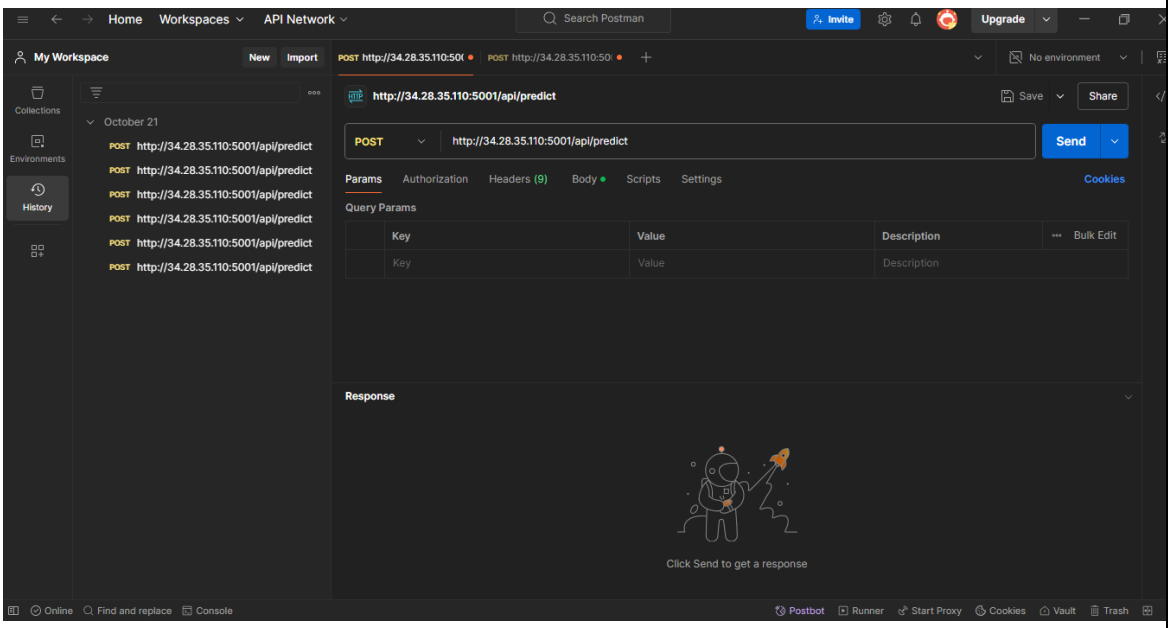
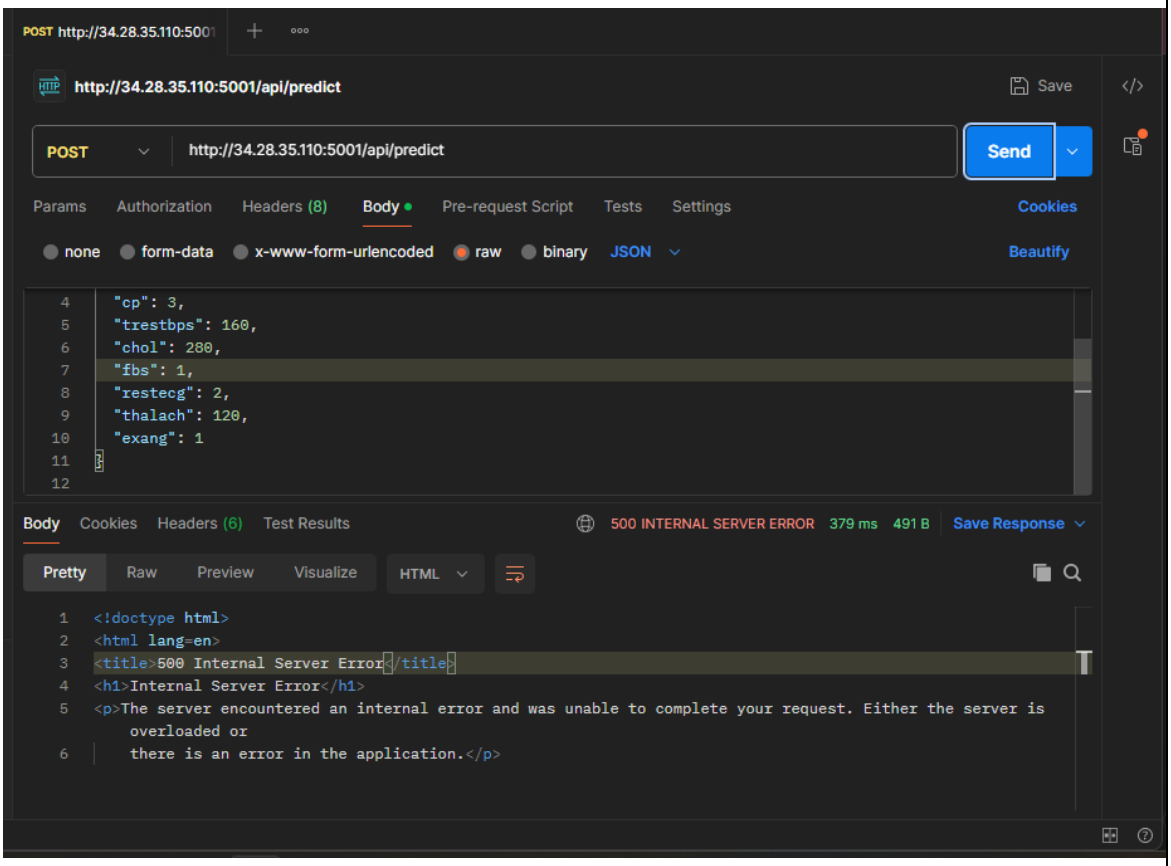
Postman

En el proyecto MedHeart, hemos decidido utilizar Postman como nuestra herramienta principal para la creación, prueba y validación de las APIs que conectan el frontend con el backend, debido a su flexibilidad y facilidad para gestionar solicitudes HTTP. Postman es esencial para asegurarnos de que la API de diagnóstico que hemos desarrollado en Flask funcione correctamente y entregue los resultados esperados al recibir los datos clínicos de los usuarios.

Postman nos permite enviar solicitudes a nuestra API, como solicitudes GET y POST, simulando el comportamiento de un usuario real. Esto es particularmente útil durante el desarrollo del proyecto, ya que nos permite verificar que la lógica de diagnóstico basada en el modelo de Random Forest esté funcionando de manera adecuada antes de integrarla completamente con el frontend de la aplicación. Al poder probar cada endpoint de la API de forma independiente, podemos identificar y corregir errores de manera rápida y eficiente.

Otra razón clave para usar Postman es su capacidad para automatizar pruebas. Podemos crear suites de pruebas para validar que cada cambio en la API sigue cumpliendo con los requisitos de negocio, lo que garantiza que el proyecto MedHeart mantenga una alta calidad y que no se introduzcan regresiones en el código. Además, la herramienta facilita la creación de diferentes entornos (desarrollo, pruebas y producción), lo que es crucial para hacer pruebas en entornos aislados y seguros.

Postman también nos permite documentar nuestras APIs de manera clara y ordenada. Esto es importante para que todo el equipo de desarrollo pueda acceder fácilmente a la información sobre cómo interactuar con la API y qué datos son necesarios en cada solicitud, lo que mejora la colaboración y facilita la integración entre el frontend y el Backend.

El uso de Postman en MedHeart ha sido fundamental para asegurar que nuestra API funcione correctamente, permitiéndonos realizar pruebas exhaustivas, automatizar validaciones y documentar eficientemente todo el flujo de interacción entre los componentes del sistema. Esto garantiza que el proceso de diagnóstico en la aplicación sea fiable, rápido y seguro.

Evidencia Postman